

Guide Détaillé v3 — Sahar

Docker complet + Agent 1 (détection Zephyr) + Agent 4 (RAG) + support Yocto

Ce qui change par rapport à ta version précédente

- L'Agent 1 ne détecte plus un projet Arduino, mais un projet **Zephyr RTOS** (présence de `prj.conf`, `CMakeLists.txt`).
 - L'Agent 4 (RAG) doit maintenant intégrer la documentation Zephyr en plus de Docker/MQTT.
 - Tu apportes un **support ponctuel** à Ameni en semaine 5-6 pour intégrer Docker dans son image Yocto.
 - Le reste (infra Docker complète, CI/CD transféré à Ameni) ne change pas.
-

Semaine 1 — Docker de base + FastAPI + premier LLM

(Identique à ta version précédente : Dockerfile backend, squelette docker-compose.yml, test_llm.py.)

Semaine 2 — Mosquitto

(Identique : ajout du service Mosquitto au docker-compose.yml, réseau Docker.)

Semaine 3 — InfluxDB + Grafana (infra)

(Identique : tu livres l'infrastructure, Nour et Ameni y branchent leurs données/dashboards.)

Semaine 4 — Agent 1 (détection Zephyr)

⚠ Ce que cette section couvre vraiment : la version "vérifie 2 fichiers" est le point de départ (10 minutes de code), pas l'objectif. Le vrai travail de la semaine, c'est de rendre ça robuste face à de vrais dépôts GitHub désordonnés. Voici à quoi ressemble concrètement ta semaine.

Jour 1 — Clonage robuste (pas juste `Repo.clone_from`)

```
python
```

```

from git import Repo, GitCommandError
import tempfile, shutil

def cloner_depot(url_github: str) -> dict:
    dossier_temp = tempfile.mkdtemp()
    try:
        Repo.clone_from(url_github, dossier_temp, depth=1) # depth=1 : clone r
        return {"succes": True, "dossier": dossier_temp}
    except GitCommandError as e:
        shutil.rmtree(dossier_temp, ignore_errors=True)
        return {"succes": False, "erreur": f"Impossible de cloner : {str(e)}"}

```

Ce que tu apprends concrètement ce jour-là : gérer une URL invalide, un dépôt privé, un timeout réseau — teste volontairement avec une mauvaise URL pour voir l'erreur gérée proprement (pas un crash).

Jour 2 — Trouver le VRAI point d'entrée (pas juste "le fichier existe")

```

python

import os

def trouver_fichiers_zephyr(dossier: str) -> dict:
    candidats_prj_conf = []
    candidats_cmake = []

    for racine, dossiers, fichiers in os.walk(dossier):
        # on ignore les dossiers de dépendances, sinon on trouve des faux posit
        if "deps" in racine or "modules" in racine or ".git" in racine:
            continue
        if "prj.conf" in fichiers:
            candidats_prj_conf.append(os.path.join(racine, "prj.conf"))
        if "CMakeLists.txt" in fichiers:
            candidats_cmake.append(os.path.join(racine, "CMakeLists.txt"))

    return {"prj_conf": candidats_prj_conf, "cmake": candidats_cmake}

```

Ce que tu apprends concrètement : pourquoi ignorer certains dossiers, comment un vrai projet peut avoir plusieurs fichiers du même nom, comment choisir le bon.

Jour 3 — Lire le contenu, pas juste vérifier l'existence

```

python

```

```
def valider_prj_conf(chemin: str) -> dict:
    with open(chemin, "r", errors="ignore") as f:
        contenu = f.read()

    if len(contenu.strip()) == 0:
        return {"valide": False, "raison": "fichier vide"}

    protocoles = []
    if "CONFIG_MQTT" in contenu:
        protocoles.append("MQTT")
    if "CONFIG_WIFI" in contenu or "CONFIG_NET_L2_WIFI" in contenu:
        protocoles.append("WiFi")

    return {"valide": True, "protocoles_detectes": protocoles}
```

Ce que tu apprends concrètement : extraire de la vraie information utile (pas juste oui/non), en cherchant des mots-clés précis dans le contenu réel du fichier.

Jour 4 — Le LLM seulement quand c'est vraiment ambigu

python

```

import anthropic, json

client = anthropic.Anthropic(api_key="TA_CLE_API")

def analyser_depot_complet(url_github: str) -> dict:
    clonage = cloner_depot(url_github)
    if not clonage["succes"]:
        return {"erreur": clonage["erreur"]}

    fichiers = trouver_fichiers_zephyr(clonage["dossier"])

    # Cas clair : pas besoin de LLM
    if fichiers["prj_conf"] and fichiers["cmake"]:
        validation = valider_prj_conf(fichiers["prj_conf"][0])
        if validation["valide"]:
            return {
                "framework": "Zephyr RTOS",
                "confiance": "haute",
                "protocoles": validation["protocoles_detectes"]
            }

    # Cas ambigu : ici, et seulement ici, on appelle le LLM
    liste_fichiers = os.listdir(clonage["dossier"])
    prompt = f"""
    Voici la liste des fichiers à la racine d'un projet embarqué : {liste_fichi
    Ce projet semble avoir une structure ambiguë (ni clairement Zephyr, ni clai
    Réponds en JSON : {{ "framework": "...", "confiance": "moyenne", "raisonneme
    """
    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=300,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.content[0].text)

```

Ce que tu apprends concrètement : la vraie compétence "agentic AI" — décider QUAND appeler un LLM (coûteux, plus lent, parfois imprécis) plutôt que de l'utiliser partout par facilité.

Jour 5 — Tester sur de VRAIS dépôts variés (pas juste 1)

Cherche sur GitHub 4-5 projets Zephyr réels et différents (exemples officiels, projets communautaires), et fais tourner ton `analyser_depot_complet()` sur chacun. Note dans un tableau ce qui marche et ce qui casse :

Dépôt testé	Résultat attendu	Résultat obtenu	Problème rencontré
<code>zephyrproject-rtos/example-application</code>	Zephyr, haute confiance	...	...
...	...	...	...

C'est cette étape de test sur des cas réels et variés qui prend le plus de temps — et c'est normal, c'est le vrai travail d'ingénierie.

Livrable de fin de semaine : publie `mock_agent1_output.json` pour Ameni dès que le format est stable :

```
json
{
  "framework": "Zephyr RTOS",
  "fichiers_detectes": ["prj.conf", "CMakeLists.txt"],
  "carte_cible": "esp32",
  "protocoles": ["MQTT", "WiFi"]
}
```

Semaine 5 — Consolidation Docker + démarrage de la base RAG

(Identique à ta version précédente : nettoyage de l'infra, scan Trivy en bonus, début de collecte documentaire pour l'Agent 4.)

Nouveau pour cette version : commence aussi à rassembler la **documentation officielle de Zephyr** (site docs.zephyrproject.org, section "Getting Started" et "Samples") comme source pour ton Agent 4 — c'est le bon moment pour la télécharger/extraire pendant que ta semaine est plus légère.

Semaine 6 — Agent 4 (RAG) + support Docker-dans-Yocto

Partie A — Agent 4 (RAG), logique inchangée mais base documentaire élargie

```
python
```

```

import chromadb
from langchain_text_splitters import RecursiveCharacterTextSplitter

client_chroma = chromadb.PersistentClient(path="./rag_db")
collection = client_chroma.get_or_create_collection("docs_techniques")
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)

def indexer_document(chemin_fichier: str, source: str):
    with open(chemin_fichier, "r") as f:
        texte = f.read()
        chunks = splitter.split_text(texte)
        ids = [f"{source}_{i}" for i in range(len(chunks))]
        collection.add(documents=chunks, ids=ids, metadatas=[{"source": source} for

# Sources à indexer :
indexer_document("docs/erreurs_docker.md", "erreurs_docker")
indexer_document("docs/erreurs_mqtt.md", "erreurs_mqtt")
indexer_document("docs/zephyr_getting_started.md", "zephyr_docs")
indexer_document("docs/erreurs_build_zephyr.md", "erreurs_zephyr") # rempli p

```

python

```

def demander_a_assistant(question: str) -> str:
    resultats = collection.query(query_texts=[question], n_results=3)
    chunks_pertinents = "\n---\n".join(resultats["documents"][0])
    prompt = f"""
    Extraits de documentation pertinents :
    {chunks_pertinents}

    Question : {question}

    Réponds en te basant sur ces extraits. Si l'information n'y est pas,
    dis-le clairement plutôt que d'inventer.
    """
    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text

```

Tester avec un cas réel Zephyr :

python

```
print(demander_a_assistant("L'erreur 'west: command not found' apparaît lors du
```

Partie B — Upgrade : raisonnement multi-étapes (inspiré de l'approche "maintenance industrielle")

💡 **Pourquoi cette amélioration :** au lieu de juste répondre à une question, l'agent suit un vrai raisonnement en plusieurs étapes, comme le ferait un technicien de maintenance. C'est gratuit à mettre en place (aucune nouvelle techno) et ça rend ta démo bien plus impressionnante.

python

```

def raisonnement_diagnostic(mesure: dict, seuils: dict) -> dict:
    """
    mesure : ex. {"temperature": 82, "capteur": "moteur_1"}
    seuils : ex. {"temperature_max": 70}
    """
    # Étape 1 – Détection d'anomalie
    anomalie_detectee = mesure["temperature"] > seuils["temperature_max"]
    if not anomalie_detectee:
        return {"statut": "normal", "diagnostic": None}

    # Étape 2-3 – Recherche documentaire + récupération des limites
    resultats = collection.query(
        query_texts=[f"limite de température recommandée pour {mesure['capteur']}"],
        n_results=3
    )
    chunks_pertinents = "\n---\n".join(resultats["documents"][0])

    # Étape 4-6 – Comparaison, diagnostic, recommandation (le LLM fait ces 3 étapes)
    prompt = f"""
    Un capteur a mesuré : {mesure}
    Le seuil normal est : {seuils}
    Voici des extraits de documentation pertinents :
    {chunks_pertinents}

    En te basant sur ces informations :
    1. Explique pourquoi cette valeur est anormale
    2. Propose une recommandation de maintenance concrète

    Réponds en JSON avec les champs "diagnostic" et "recommandation".
    """
    response = client.messages.create(
        model="claude-sonnet-4-5", max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
    )
    resultat = json.loads(response.content[0].text)

    # Étape 7 – Notification (ici : simple enregistrement, pas besoin de vrai service)
    resultat["ticket_cree"] = True
    resultat["horodatage"] = str(datetime.now())
    with open("tickets/tickets.jsonl", "a") as f:
        f.write(json.dumps(resultat) + "\n")

    return resultat

```

Tester:


```
python
```

```
print(raisonnement_diagnostic(  
    mesure={"temperature": 82, "capteur": "esp32_salon"},  
    seuils={"temperature_max": 70}  
))
```

Livable : l'agent détecte une anomalie, cherche la documentation pertinente, génère un diagnostic ET une recommandation, et enregistre un "ticket" — les 7 étapes en une seule fonction, sans matériel ni bibliothèque supplémentaire.

Partie C — Support ponctuel à Ameni (Docker dans Yocto)

■ **Intégrer Docker dans une image Yocto :** ajouter une "recette" BitBake qui installe le moteur Docker à l'intérieur de l'image Linux générée, pour que le Raspberry Pi ait Docker disponible dès son premier démarrage.

Ton rôle ici n'est **pas de maîtriser Yocto toi-même**, mais d'apporter ton expertise Docker :

- Vérifie avec Ameni que la version de Docker Engine choisie pour la recette Yocto est compatible avec tes images (celles de ton `docker-compose.yml`).
- Aide à tester que les conteneurs Mosquitto/InfluxDB/Grafana démarrent correctement une fois l'image Yocto lancée (dans QEMU ou sur le vrai Raspberry Pi).

Livable : l'Agent 4 répond correctement sur au moins 3 cas d'erreurs réels (Docker + Zephyr) + validation que ton infra Docker tourne dans l'environnement d'Ameni.

Semaine 7 — Intégration finale

Chaîne complète : GitHub → Agent 1 (toi) → Agent 2 (Ameni) → Agent 3 (Nour) → Build Runner (Ameni) → Déploiement sur passerelle (Yocto ou Raspberry Pi OS standard) → Monitoring → Diagnostic RAG (toi).

Ta soupape de sécurité (mise à jour)

1. Sacrifie en premier : le support Yocto de la semaine 6 (Ameni peut avancer seule si tu es débordée par le RAG).
 2. Sacrifie ensuite : l'indexation de la doc Zephyr complète dans le RAG — garde au moins les erreurs de build les plus fréquentes.
 3. Ne sacrifie jamais : la publication de `mock_agent1_output.json`.
-

Combine ce document avec le cahier des charges fusionné pour la vue d'ensemble.